

HARDWARE DESIGN
RTL IMPLEMENTATION
SYSTEM VERIFICATION

SERIAL
COMMUNICATION
MAKES
BIG THINGS
POSSIBLE

UART CONTROLLER

DESIGN | IMPLEMENTATION | VERIFICATION

A SYNTHESIZABLE UART CONTROLLER WITH FIFOS,
CONFIGURABLE BAUD RATE AND FULL-DUPLEX COMMUNICATION



TRANSMIT



RECEIVE



CONFIGURE



COMMUNICATE

RELIABLE
FLEXIBLE
EFFICIENT



Prepared by
Ahmed Emad Atris Elsayed

CIRCUITS | LOGIC | COMMUNICATION | REAL IMPACT

SMALL SIGNALS
BIG CONNECTIONS

Summary

This project presents the design and implementation of a synthesizable UART Controller using Verilog HDL. The controller is designed to provide reliable serial communication while integrating data buffering, programmable baud-rate control, and a memory-mapped register interface within a single hardware peripheral.

The design supports the standard 8-N-1 UART communication format, including one start bit, eight data bits transmitted LSB first, and one stop bit. Separate 16×8 TX and RX FIFOs are used to buffer transmitted and received data, allowing the UART interface to handle data independently from the host-side register accesses.

The controller is organized into several RTL modules, including the UART transmitter, UART receiver, TX/RX FIFOs, baud-rate generator, register interface, and top-level integration module. The host can configure the UART through the CONTROL and BAUD_DIV registers and monitor its operation through the STATUS register.

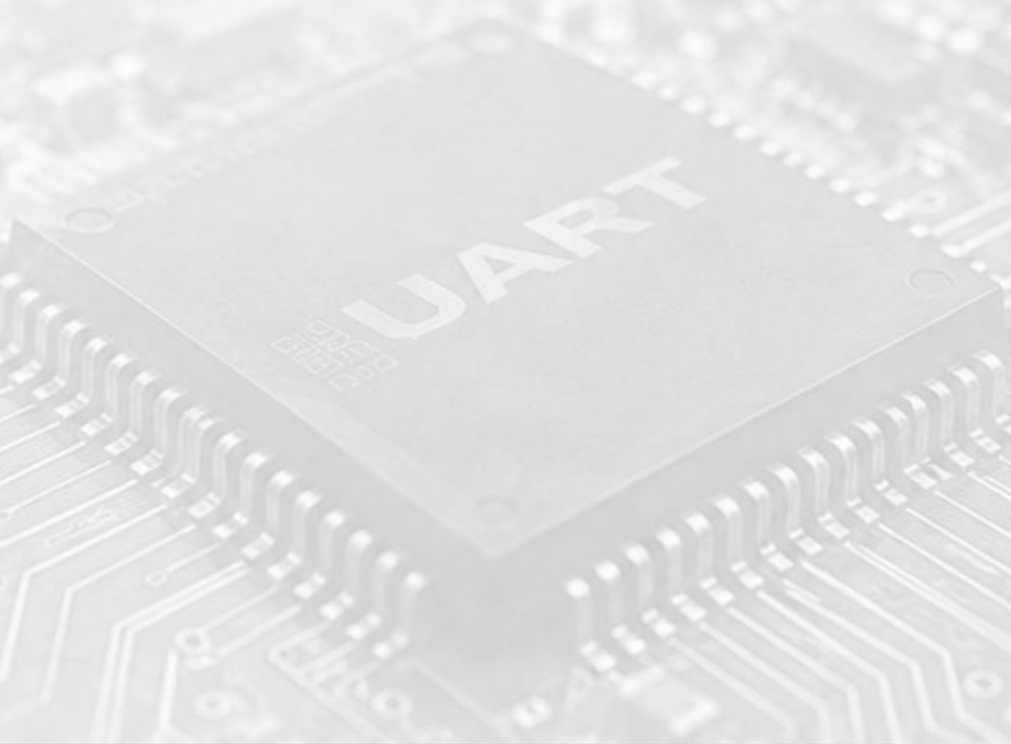
The UART receiver uses 16× oversampling to detect and sample incoming data, while the transmitter serializes each byte according to the defined UART frame format. The receiver also checks the stop bit and generates a frame-error indication when an invalid stop bit is detected.

The complete design was verified using Xilinx Vivado through a dedicated testbench containing eleven functional tests. The verification covers reset operation, single-byte and multiple-byte transmission and reception, FIFO full conditions, bad stop-bit detection, register access, baud-rate scaling, and full-duplex loopback communication. All defined verification tests passed successfully.

Table of Contents

Project Overview	5
System Architecture	6
UART Protocol	7
3.1 Baud Rate and Timing	8
Register Interface	9
4.1 Register Map	9
4.2 TX_DATA Register	9
4.3 RX_DATA Register	10
4.4 STATUS Register	10
4.5 CONTROL Register	10
4.6 BAUD_DIV Register	11
4.7 Register Interface Summary	11
RTL Module Implementation	12
5.1 FIFO Module	12
5.1.1 FIFO Internal Structure	12
5.2 Baud Rate Generator	13
5.3 UART Transmitter	13
5.3.1 UART TX State Machine	14
5.4 UART Receiver	14
5.4.1 UART RX State Machine	15
5.5 UART Register Interface Module	15
5.6 Testbench	16
5.7 Top-Level Module	17
Functional Verification	18
6.1 Verification Test Plan	18
6.2 T1 — Reset Test	18
6.2 T2 — Single-Byte TX	19
6.3 T3 — Multiple-Byte TX	20
6.4 T4 — TX FIFO Full	20

6.5 T5 — Single-Byte RX	21
6.6 T6 — Multiple-Byte RX	22
6.7 T7 — RX FIFO Full	22
6.8 T8 — Bad Stop Bit	23
6.9 T9 — Register Access	24
6.10 T10 — Baud Scaling	24
6.11 T11 — Full-Duplex Loopback	25
RTL Schematic	28
Conclusion and Future Improvements	29



Project Overview

The UART Controller is a synthesizable digital communication peripheral designed to provide serial data transmission and reception through a standard UART interface.

The controller integrates the main hardware blocks required for UART communication into a single RTL design. These blocks include a UART transmitter, UART receiver, independent TX and RX FIFOs, a programmable baud-rate generator, and a memory-mapped register interface.

The communication protocol follows the **8-N-1 UART format**, where each transmitted frame consists of one start bit, eight data bits transmitted LSB first, and one stop bit. The TX and RX paths operate independently, allowing data to be transmitted and received through separate hardware paths.

The host interacts with the controller through a set of memory-mapped registers. Data can be written to the TX FIFO through the TX_DATA register, while received data can be retrieved from the RX FIFO through the RX_DATA register. Configuration and monitoring are handled through the CONTROL, STATUS, and BAUD_DIV registers.

The design was implemented using modular Verilog RTL, with each functional block developed and verified independently before being integrated at the top level. The complete controller was then evaluated using a dedicated simulation testbench covering the defined functional requirements.

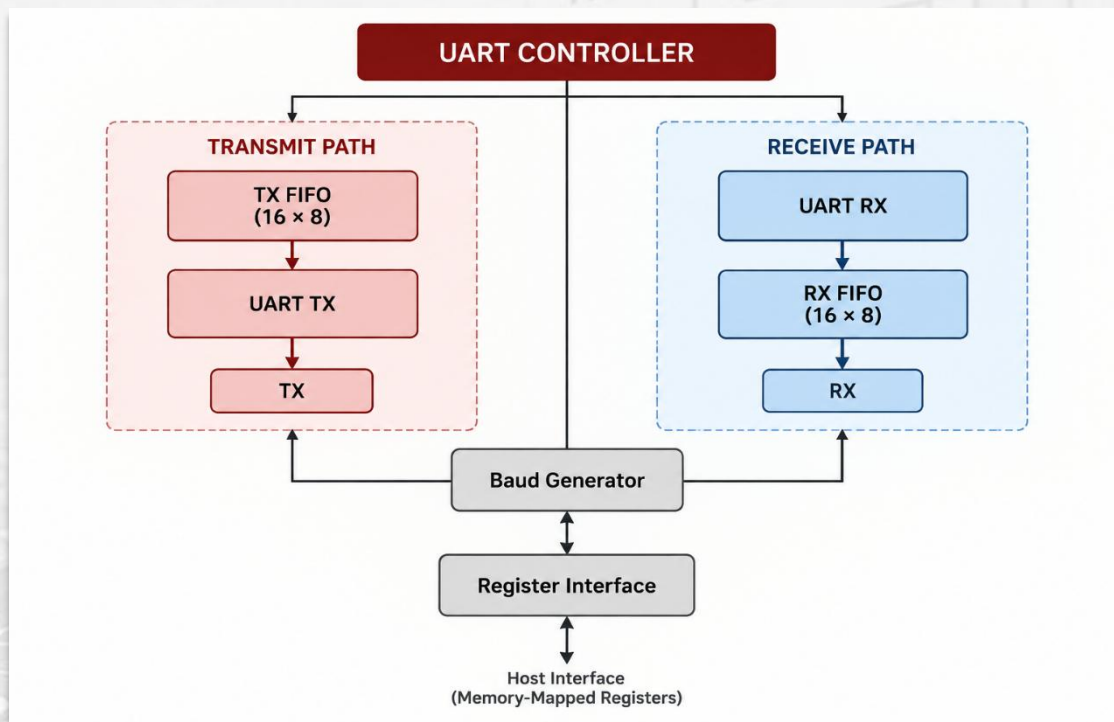


Figure 2.1: High-Level Architecture of the UART Controller

System Architecture

The UART Controller is implemented as a modular RTL design in which each hardware block is responsible for a specific part of the communication process. The architecture is divided into the transmit path, receive path, timing generation, and host-side register interface.

The transmit and receive paths operate independently. The transmit path handles data written by the host, stores it in the TX FIFO, and serializes each byte through the UART transmitter. The receive path monitors the external RX signal, reconstructs incoming UART frames, and stores valid received bytes in the RX FIFO for later access by the host.

The controller uses a programmable baud-rate generator to provide the timing pulses required by both UART TX and UART RX. The baud timing is controlled through the BAUD_DIV register, allowing the communication speed to be changed through the register interface.

The register interface connects the host-side bus signals to the internal UART resources. It handles register address decoding, read and write operations, UART configuration, transmit data insertion, received data retrieval, and status monitoring.

All major functional blocks are implemented as independent RTL modules and are connected through the top-level uart_top module. This structure keeps the communication logic, buffering, timing, and control functions separated, making the design easier to verify and integrate.

Module	Main Function
uart_top	Top-level integration of all UART blocks
uart_regs	Register decoding, configuration, and host interface
FIFO	16×8 buffering for TX and RX data
baud_gen	Generates programmable UART timing pulses
uart_tx	Serializes and transmits 8-N-1 frames
uart_rx	Receives and reconstructs 8-N-1 frames
uart_tb	Functional verification of the complete controller

Table 2.1: Main RTL Modules of the UART Controller

UART Protocol

The UART Controller uses a standard **8-N-1 UART** communication format for serial data transmission and reception. The communication interface operates asynchronously, meaning that the transmitter and receiver do not share a dedicated clock signal. Instead, both sides rely on the configured baud rate to determine the timing of each transmitted and received bit.

When the UART interface is idle, the TX and RX lines remain at logic HIGH. A transmission begins with a START bit, which drives the line LOW for one bit period. This is followed by eight data bits, transmitted from D0 to D7 (LSB first). The frame is completed by a STOP bit, which drives the line HIGH for one bit period.

No parity bit is used in the implemented Phase 1 design. Therefore, each complete UART frame contains a total of 10 bits: one start bit, eight data bits, and one stop bit.

On the receive side, the UART RX module detects the falling edge of the start bit and uses $16\times$ oversampling to determine the sampling points for the incoming data. The data bits are reconstructed sequentially from D0 through D7. After receiving all eight data bits, the receiver samples the stop bit. A HIGH stop bit indicates a valid frame, while a LOW stop bit generates a frame-error indication and the received byte is discarded.

UART 8-N-1 FRAME

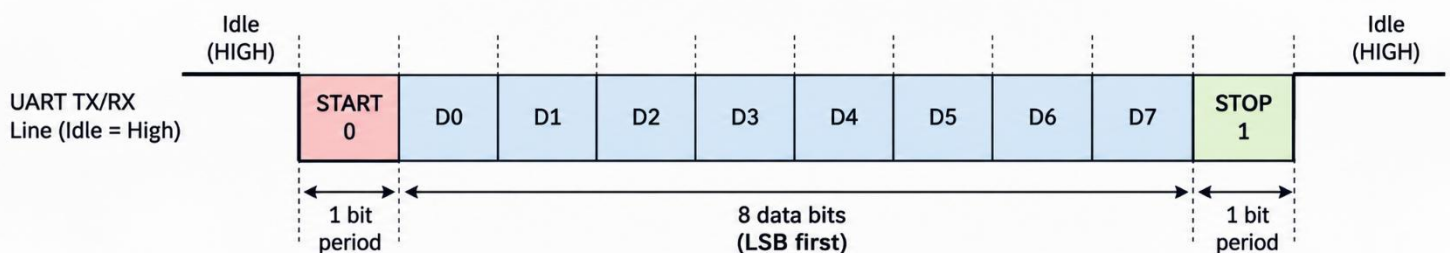


Figure 3.1: UART 8-N-1 Frame Format

3.1 Baud Rate and Timing

The timing of the UART communication is controlled by the programmable **BAUD_DIV** value. The baud generator uses the system clock and the configured divisor to generate periodic timing pulses for the UART transmitter and receiver.

In the implemented design, the baud generator produces one timing pulse every:

$\text{BAUD_DIV} \times 16$ system clock cycles

The factor of 16 corresponds to the receiver oversampling rate. The same generated timing reference is used by the TX and RX modules to maintain consistent communication timing.

Changing the **BAUD_DIV** value directly changes the duration of each UART bit. Therefore, increasing the divisor increases the bit period, while decreasing the divisor reduces it.



Register Interface

The UART Controller provides a memory-mapped register interface that allows the host to access transmit and receive data, configure the UART operation, monitor its status, and program the baud-rate divisor.

The register interface uses a 4-bit address bus and supports separate read and write control signals. Each register is assigned a fixed address, while unsupported or unused addresses return zero during read operations.

The interface contains five defined registers: TX_DATA, RX_DATA, STATUS, CONTROL, and BAUD_DIV. TX_DATA is used to write bytes into the transmit FIFO, while RX_DATA provides access to the oldest byte stored in the receive FIFO. The STATUS register reports the current state of the TX and RX paths, including FIFO status, transmitter activity, received-data availability, and frame errors.

The CONTROL register provides independent enable signals for the UART, transmitter, and receiver. The BAUD_DIV register stores the programmable divisor used by the baud-rate generator. A non-zero divisor is required for valid baud-rate generation.

4.1 Register Map

Address	Register	Access	Reset Value	Description
0x0	TX_DATA	W	0x00	Writes a byte to the TX FIFO
0x4	RX_DATA	R	0x00	Reads the oldest byte from the RX FIFO
0x8	STATUS	R	0x14	Reports UART and FIFO status
0xC	CONTROL	R/W	0x07	Controls UART, TX, and RX enables
0xD	BAUD_DIV	R/W	0x1B	Programmable baud-rate divisor

Table 4.1: UART Register Map

4.2 TX_DATA Register

The TX_DATA register is a write-only register located at address 0x0. When a valid write operation is performed and the TX FIFO is not full, the provided byte is written into the TX FIFO. If the TX FIFO is full, the write operation is rejected to prevent data overflow.

4.3 RX_DATA Register

The RX_DATA register is a read-only register located at address 0x4. A valid read operation retrieves the oldest byte available in the RX FIFO and generates a FIFO read request. If the RX FIFO is empty, the register returns 0x00.

4.4 STATUS Register

Bit	Name	Description
0	TX_BUSY	Indicates that the UART transmitter is active
1	TX_FIFO_FULL	TX FIFO is full
2	TX_FIFO_EMPTY	TX FIFO is empty
3	RX_FIFO_FULL	RX FIFO is full
4	RX_FIFO_EMPTY	RX FIFO is empty
5	RX_AVAILABLE	Indicates that received data is available
6	RX_FRAME_ERROR	Indicates an invalid UART stop bit
7	Reserved	Tied to 0

The STATUS register is read-only and provides real-time information about the state of the UART transmitter, TX FIFO, RX FIFO, and receiver error status.

4.5 CONTROL Register

Bit	Name	Description
0	UART_ENABLE	Enables the UART controller
1	TX_ENABLE	Enables UART transmission
2	RX_ENABLE	Enables UART reception
7:3	Reserved	Read as 0

The CONTROL register allows the host to independently enable the UART controller, transmitter, and receiver. Its reset value is 0x07, enabling all three functions by default.

4.6 BAUD_DIV Register

The BAUD_DIV register is a programmable 8-bit read/write register located at address 0xD. It controls the timing interval generated by the baud-rate generator. The register accepts non-zero divisor values, while a value of zero is rejected by the register interface.

The default reset value is 27 (0x1B). Changing the divisor changes the UART bit period proportionally, which was verified during the baud-rate scaling test.

During normal operation, the value stored in BAUD_DIV is continuously provided to the baud-rate generator. The generator uses this value to determine when a new timing pulse should be generated. These timing pulses are then used by both the transmitter and receiver to maintain the required UART timing relationship.

A larger BAUD_DIV value results in a longer interval between generated timing pulses and therefore a longer UART bit period. Conversely, reducing the divisor produces timing pulses at shorter intervals and reduces the bit period. This provides a simple programmable mechanism for controlling the communication timing without modifying the UART TX or RX logic.

4.7 Register Interface Summary

The register interface provides the complete host-side control and monitoring mechanism for the UART Controller. Through the five defined registers, the host can insert transmit data, retrieve received data, configure the UART enables, monitor FIFO and communication status, and control the baud-rate timing.

The separation between data registers and control/status registers allows the host interface to remain simple while the internal UART modules handle the timing, buffering, serialization, and frame processing independently.

Register	Main Function
TX_DATA	Provides transmit data to the TX FIFO
RX_DATA	Retrieves received data from the RX FIFO
STATUS	Monitors UART and FIFO conditions
CONTROL	Enables and controls UART operation
BAUD_DIV	Configures UART timing

Table 4.2: Register Interface Functional Summary

RTL Module Implementation

5.1 FIFO Module

The FIFO module provides temporary storage for UART transmit and receive data. Two independent instances of the same FIFO module are used in the design: one for the TX path and another for the RX path.

Each FIFO has a capacity of 16 entries, with each entry storing 8 bits, resulting in a 16×8 FIFO structure. The FIFO uses separate read and write pointers to control the memory locations and a counter to track the number of stored entries.

Data is written into the FIFO when `wr_en` is asserted and the FIFO is not full. Similarly, data is removed when `rd_en` is asserted and the FIFO is not empty. The write and read pointers advance after their corresponding operations and wrap around when reaching the end of the memory.

The FIFO continuously generates two status flags: `full_flag` and `empty_flag`. The full condition occurs when all 16 entries are occupied, while the empty condition occurs when the FIFO contains no valid data.

In the TX path, the host writes bytes into the TX FIFO through the register interface, and the UART transmitter reads them when it is ready to transmit. In the RX path, the UART receiver writes validated bytes into the RX FIFO, and the host reads them through the `RX_DATA` register.

This separation between the host and UART data paths allows the transmitter and receiver to buffer data independently and prevents direct timing dependencies between register accesses and serial communication.

5.1.1 FIFO Internal Structure

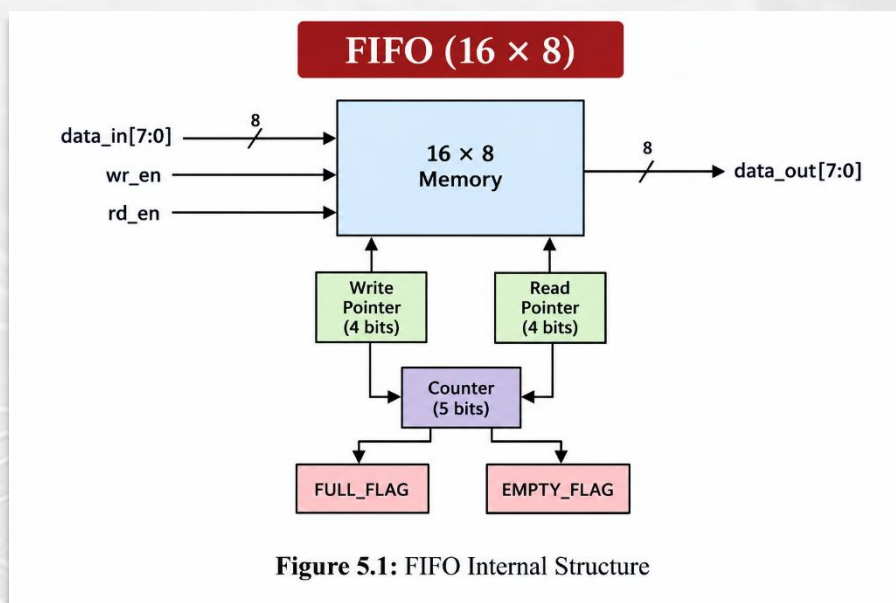


Figure 5.1: FIFO Internal Structure

Figure 5.1: FIFO Internal Structure

5.2 Baud Rate Generator

The Baud Rate Generator is responsible for generating the timing reference used by the UART transmitter and receiver. It operates synchronously with the system clock and uses the programmable BAUD_DIV value provided by the register interface.

An internal counter increments on every system-clock cycle. When the counter reaches the programmed terminal value, a one-cycle baud_tick pulse is generated and the counter is reset. The generated timing pulse is then supplied to both the UART TX and UART RX modules.

The implemented timing relationship is:

$$\text{baud tick period} = \text{BAUD_DIV} \times 16 \text{ system clock cycles}$$

The factor of 16 corresponds to the receiver's oversampling rate. Using a common timing reference allows the TX and RX modules to operate consistently with the configured baud-rate setting.

The divisor is programmable through the BAUD_DIV register, allowing the communication timing to be modified without changing the RTL structure of the transmitter or receiver.

```
begin
  counter<=counter+1;
  if(counter==(baud_div*16)-1)
    begin
      baud_tick<=1'b1;
      counter<=0;
    end
end
```

Figure 5.2: Baud Tick Generation Logic

5.3 UART Transmitter

The UART transmitter converts an 8-bit parallel data byte received from the TX FIFO into a serial UART frame. The transmitter is implemented as a finite state machine with four states: IDLE, START, DATA, and STOP.

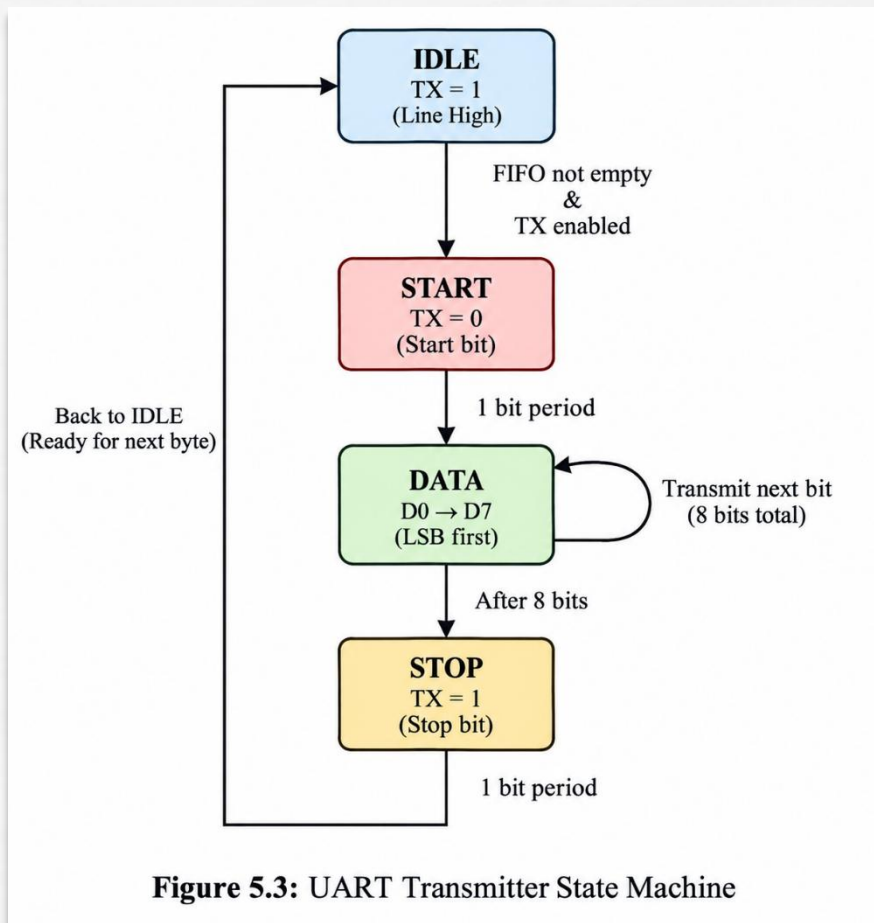
In the IDLE state, the TX line remains HIGH while the transmitter waits for a valid byte in the TX FIFO. When the UART and transmitter are enabled and the FIFO is not empty, the byte is loaded into an internal register and transmission begins.

The START state drives the TX line LOW for one bit period. The transmitter then enters the DATA state and sends the eight data bits sequentially from D0 to D7, following the required LSB-first UART format.

After the eighth data bit is transmitted, the transmitter enters the STOP state and drives the TX line HIGH for one bit period. The transmitter then returns to IDLE and becomes ready for the next byte.

The tx_busy signal indicates that a byte is currently being transmitted, while fifo_rd_en is used to retrieve the next byte from the TX FIFO.

5.3.1 UART TX State Machine



5.4 UART Receiver

The UART receiver is responsible for detecting incoming serial data, reconstructing the transmitted byte, validating the UART frame, and storing valid received data in the RX FIFO. The receiver is implemented as a finite state machine with four states: IDLE, START, DATA, and STOP.

In the IDLE state, the RX line is monitored while the receiver waits for a falling edge indicating the beginning of a UART frame. Once a valid start transition is detected and the UART and receiver are enabled, the receiver enters the START state.

During the START state, the receiver uses the generated baud timing to reach the middle of the start-bit period. The RX signal is then checked to confirm that the start bit is still LOW. If the expected start condition is not detected, the receiver returns to IDLE.

In the DATA state, the receiver samples one data bit at the center of each bit period. The received bits are stored sequentially from D0 through D7, reconstructing the original 8-bit byte in LSB-first order. After all eight data bits have been sampled, the receiver moves to the STOP state.

In the STOP state, the receiver checks the stop bit at the expected sampling point. A HIGH value indicates a valid UART frame, and the reconstructed byte is written to the RX FIFO. If the stop bit is LOW, the

frame is considered invalid, the received byte is discarded, and the `RX_FRAME_ERROR` indication is asserted.

The receiver therefore provides both data reconstruction and basic frame validation before a byte is made available to the host through the RX FIFO.

5.4.1 UART RX State Machine

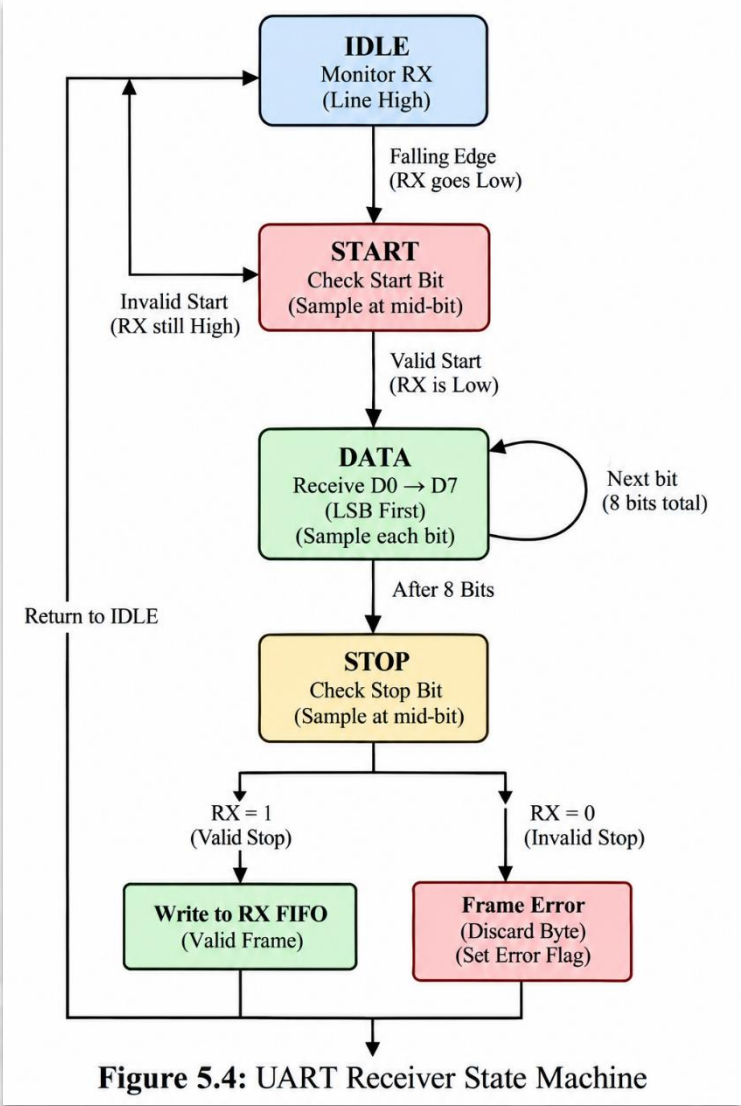


Figure 5.4: UART Receiver State Machine

5.5 UART Register Interface Module

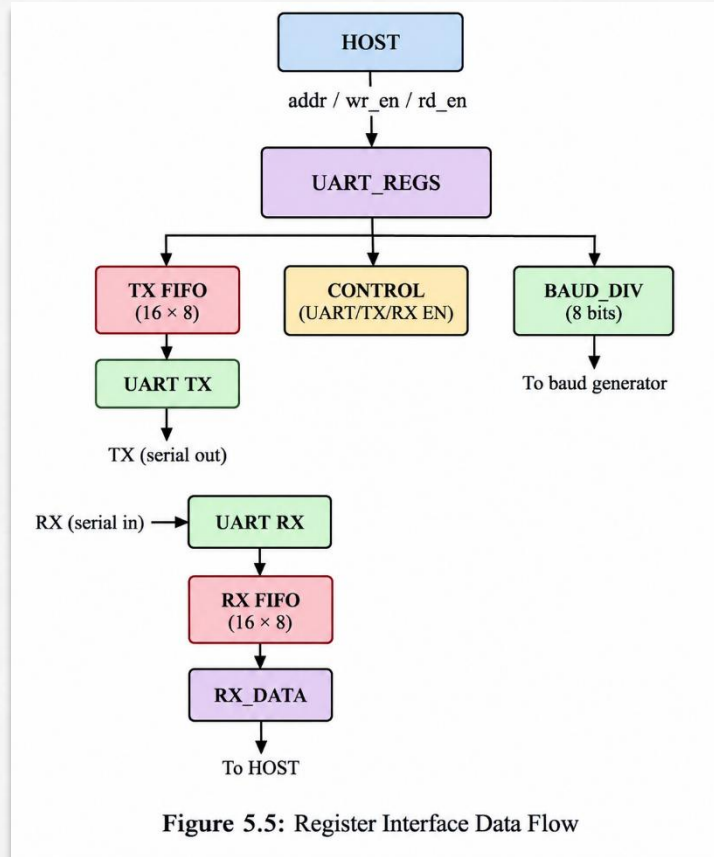
The `uart_regs` module implements the host-side register interface of the UART Controller. It decodes the incoming address and controls read and write operations for the defined UART registers.

During a `TX_DATA` write operation, the module forwards the provided byte to the TX FIFO when the FIFO is not full. During an `RX_DATA` read operation, it provides the oldest available byte from the RX FIFO and generates the corresponding FIFO read request.

The module also stores the UART configuration values in the `CONTROL` and `BAUD_DIV` registers. The `CONTROL` register provides the UART, TX, and RX enable signals, while `BAUD_DIV` provides the programmable divisor to the baud-rate generator.

The STATUS register is generated from the current state of the internal UART blocks. It reports TX activity, FIFO full and empty conditions, received-data availability, and RX frame errors.

Read data is selected according to the active register address, while write operations update only the registers that support write access.



5.6 Testbench

The `uart_tb` module is used to verify the complete UART Controller at the system level. The testbench generates the system clock and synchronous reset, applies register read and write transactions, provides UART RX stimuli, and monitors the TX and RX paths.

The verification environment includes functional tests for reset behavior, transmission, reception, FIFO boundary conditions, frame-error detection, register access, baud-rate scaling, and full-duplex communication.

Each test sets a corresponding pass flag, allowing the final simulation output to provide a clear summary of the verification results. The testbench was used to confirm the interaction between the individual RTL modules after their integration into the complete controller.

5.7 Top-Level Module

The `uart_top` module is the top-level integration module of the UART Controller. It connects the register interface, TX and RX FIFOs, UART transmitter, UART receiver, and baud-rate generator into one complete peripheral.

The TX path connects the host register interface to the TX FIFO and then to the UART transmitter. Similarly, the RX path connects the external RX input to the UART receiver, then to the RX FIFO and finally back to the host through the register interface.

The baud generator provides the common timing reference to both UART TX and UART RX, while the configuration and status signals are exchanged through the register interface.

The `irq` output is tied LOW in the Phase 1 implementation, as interrupt functionality is reserved for a future extension.



Functional Verification

6.1 Verification Test Plan

Test	Verification Case	Main Function Verified
T1	Reset	Correct reset values and initial UART state
T2	Single-Byte TX	Transmission of one UART byte
T3	Multiple-Byte TX	Sequential transmission of multiple bytes
T4	TX FIFO Full	TX FIFO capacity and full condition
T5	Single-Byte RX	Reception of one UART byte
T6	Multiple-Byte RX	Reception and buffering of multiple bytes
T7	RX FIFO Full	RX FIFO capacity and full condition
T8	Bad Stop Bit	Detection of an invalid UART stop bit
T9	Register Access	Read/write operation of UART registers
T10	Baud Scaling	Effect of BAUD_DIV on transmission timing
T11	Full-Duplex	Simultaneous TX/RX loopback communication

Table 6.1: UART Controller Verification Test Plan

6.2 T1 — Reset Test

Objective

Verify that all UART modules return to their defined reset state when rst is asserted.

Procedure

The testbench applies the synchronous active-high reset and monitors the main control, FIFO, and UART status signals.

Expected Result

The UART returns to its initial idle state, the FIFOs become empty, and the configuration registers return to their default values.

Result: PASS ✓

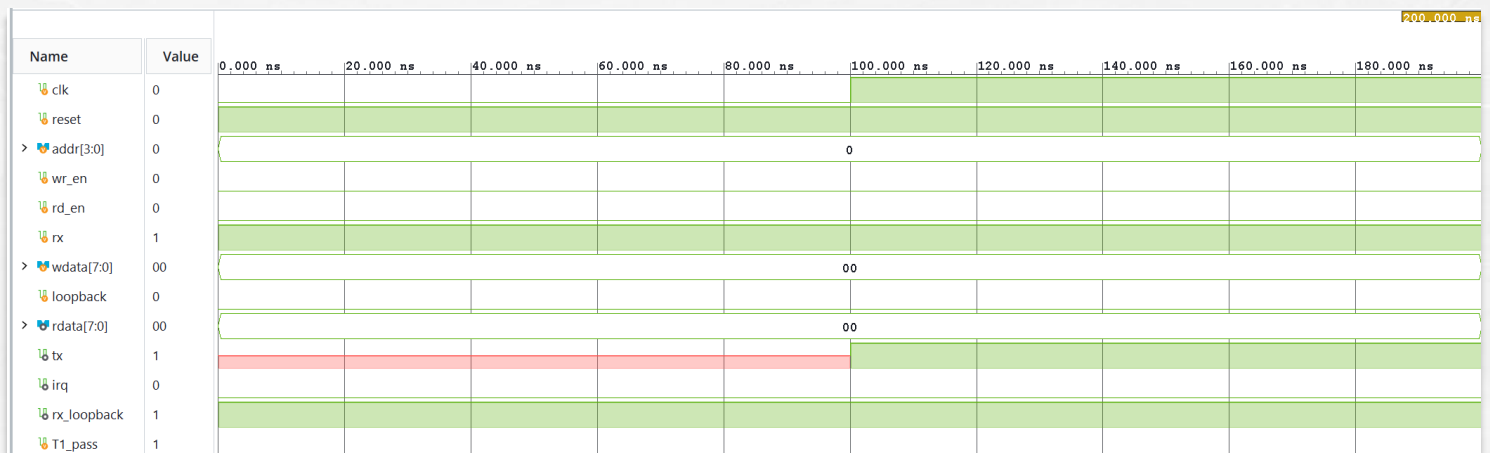


Figure 6.1: T1 Reset Test — Vivado Simulation Waveform

6.2 T2 — Single-Byte TX

Objective

Verify the transmission of a single byte through the UART TX path.

Procedure

The byte 8'hA5 is written to the TX_DATA register and transmitted through the UART TX output.

Expected Result

A valid 8-N-1 frame is generated with the data transmitted **LSB first**.

Result: PASS ✓

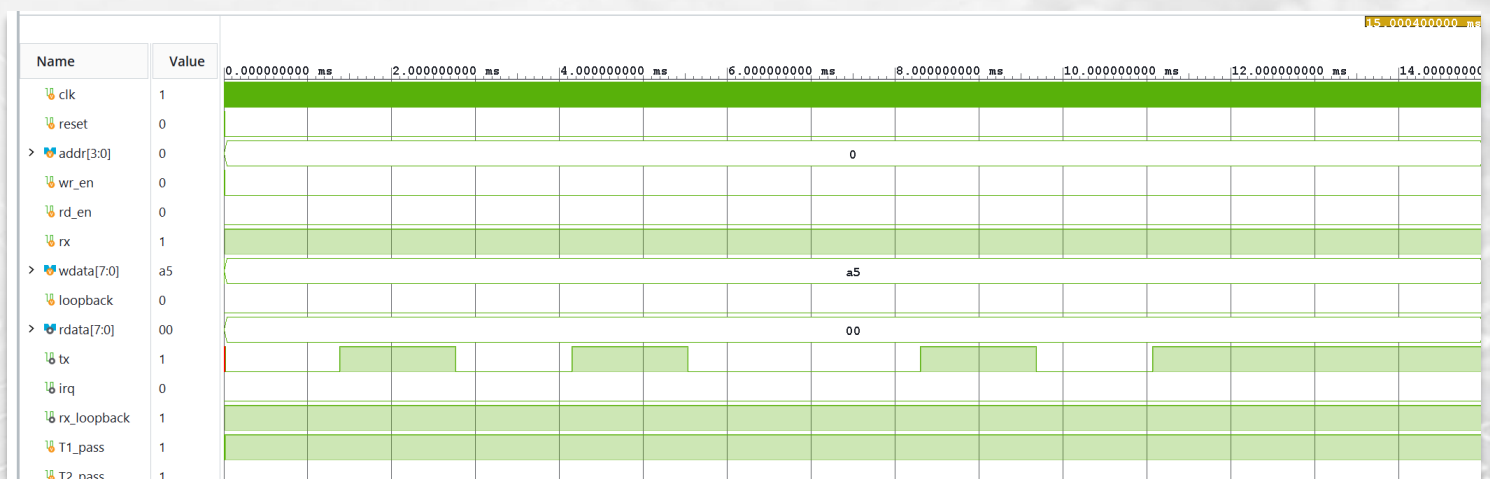


Figure 6.2: T2 Single-Byte TX — Vivado Simulation Waveform

6.3 T3 — Multiple-Byte TX

Objective

Verify the sequential transmission of multiple bytes through the UART TX path.

Procedure

Multiple data bytes are written consecutively to the TX_DATA register and transmitted through the UART TX output.

Expected Result

All bytes are transmitted correctly and sequentially without data loss.

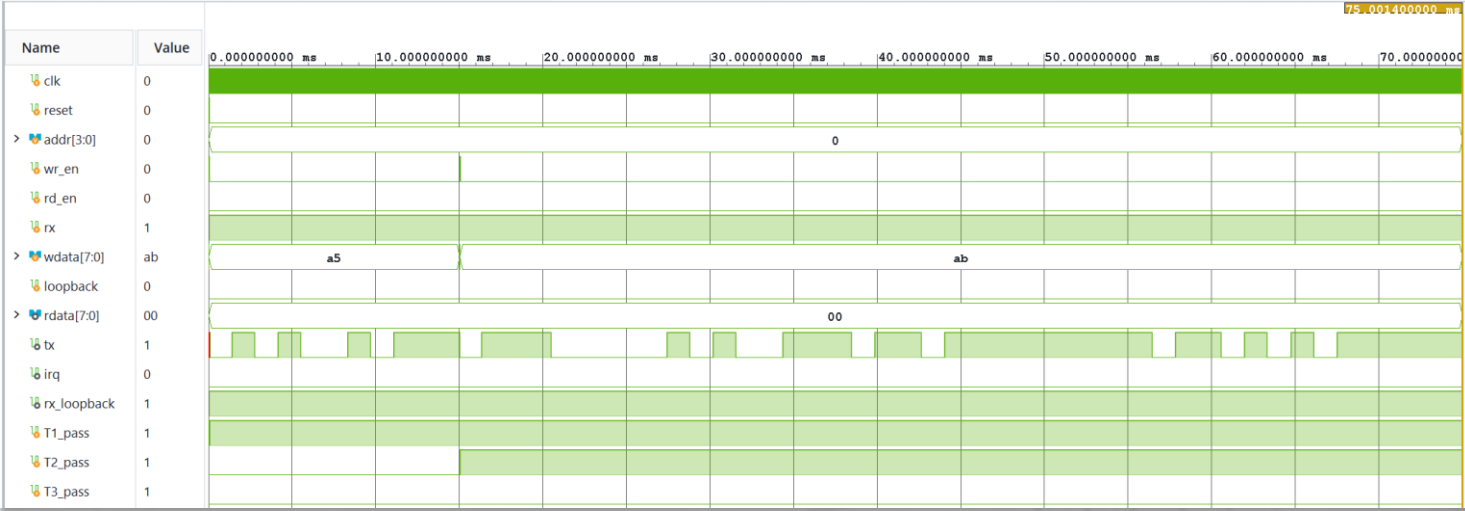


Figure 6.3 T3 Multiple-Byte TX — Vivado Simulation Waveform

6.4 T4 — TX FIFO Full

Objective

Verify that the TX FIFO correctly detects and handles the full condition.

Procedure

The TX FIFO is filled with 16 bytes until the TX_FIFO_FULL status becomes active. An additional write is then attempted.

Expected Result

The FIFO indicates the full condition and rejects the additional write without overflowing.

Result: PASS ✓

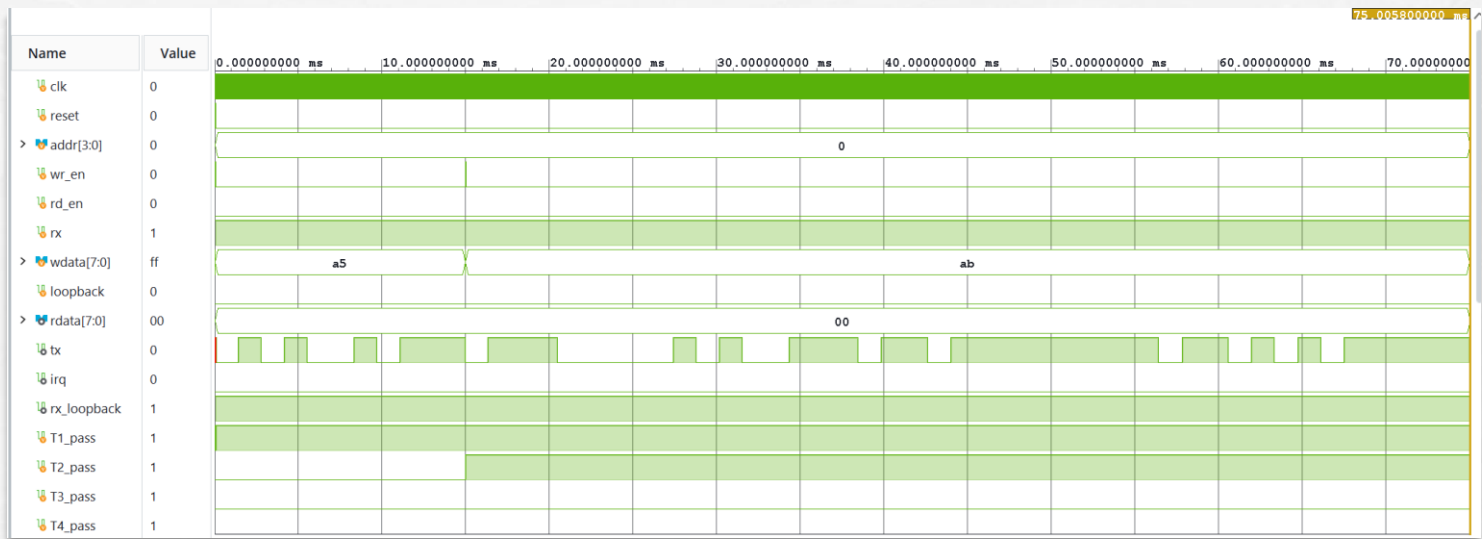


Figure 6.4: T4 TX FIFO Full — Vivado Simulation Waveform

6.5 T5 — Single-Byte RX

Objective

Verify the reception of a single UART byte through the RX path.

Procedure

A complete UART frame containing 8'hAC is applied to the RX input using the 8-N-1 format.

Expected Result

The receiver reconstructs the byte correctly and stores it in the RX FIFO for host access.

Result: PASS ✓

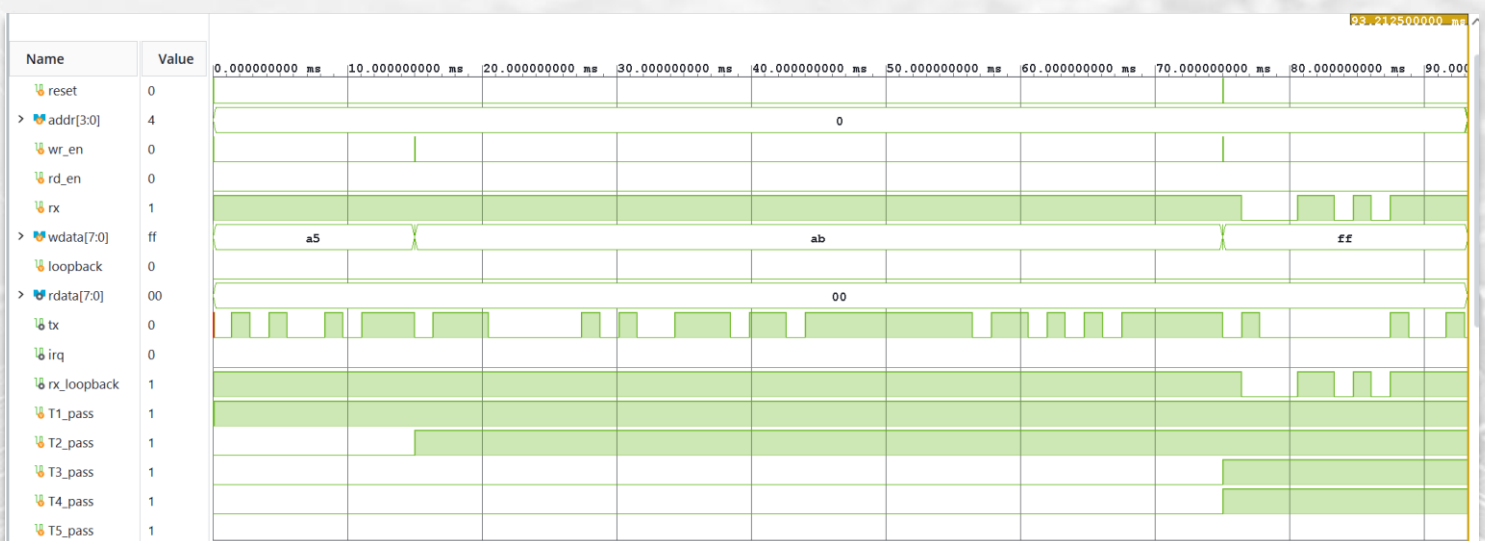


Figure 6.5: T5 Single-Byte RX — Vivado Simulation Waveform

6.6 T6 — Multiple-Byte RX

Objective

Verify reception and buffering of multiple UART bytes.

Procedure

Four UART frames containing 8'hAC, 8'h35, 8'h7F, and 8'h12 are applied sequentially to the RX input.

Expected Result

All four bytes are received correctly and stored sequentially in the RX FIFO.

Result: PASS 

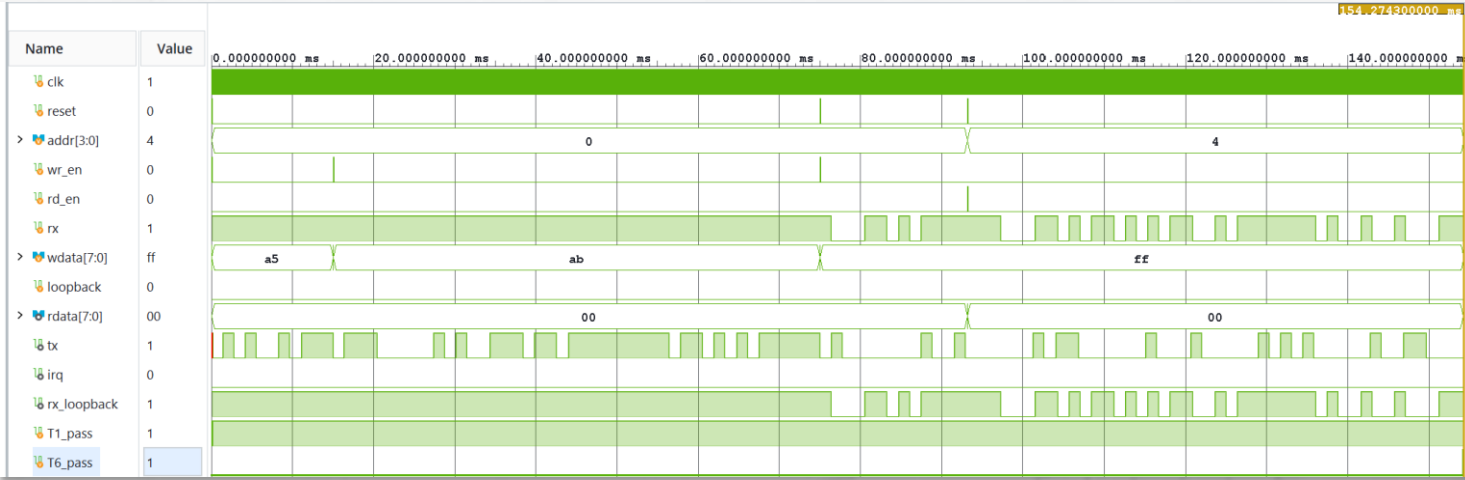


Figure 6.6: T6 Multiple-Byte RX — Vivado Simulation Waveform

6.7 T7 — RX FIFO Full

Objective

Verify the RX FIFO full condition and protection against overflow.

Procedure

Sixteen valid UART bytes are received to fill the RX FIFO completely, followed by an additional received byte.

Expected Result

The RX_FIFO_FULL flag becomes active and the additional byte is rejected without overflowing the FIFO.

Result: PASS 

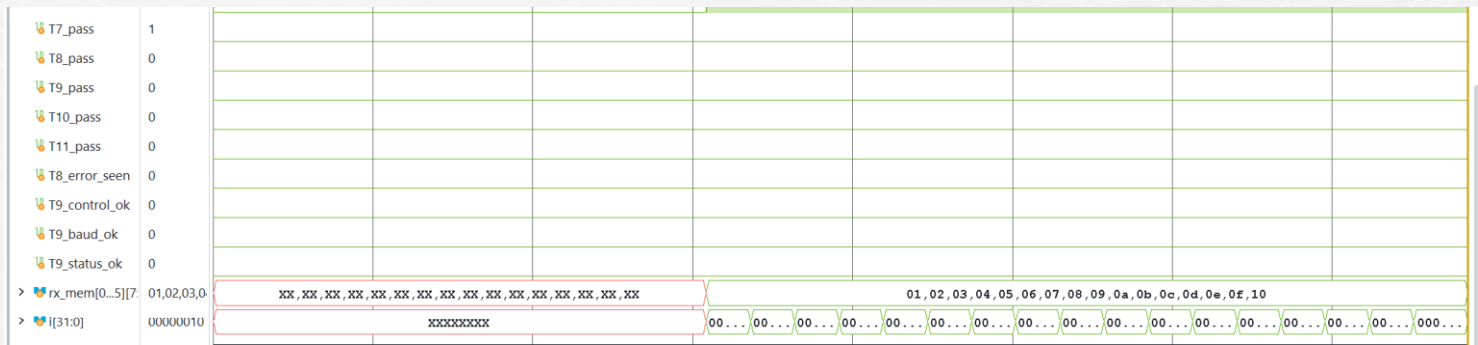


Figure 6.7: T7 RX FIFO Full — Vivado Simulation Waveform

6.8 T8 — Bad Stop Bit

Objective

Verify detection of an invalid UART stop bit.

Procedure

A UART frame is applied with the stop bit forced LOW instead of the required HIGH level.

Expected Result

The receiver detects the invalid stop bit, asserts `RX_FRAME_ERROR`, and discards the received byte.

Result: PASS 

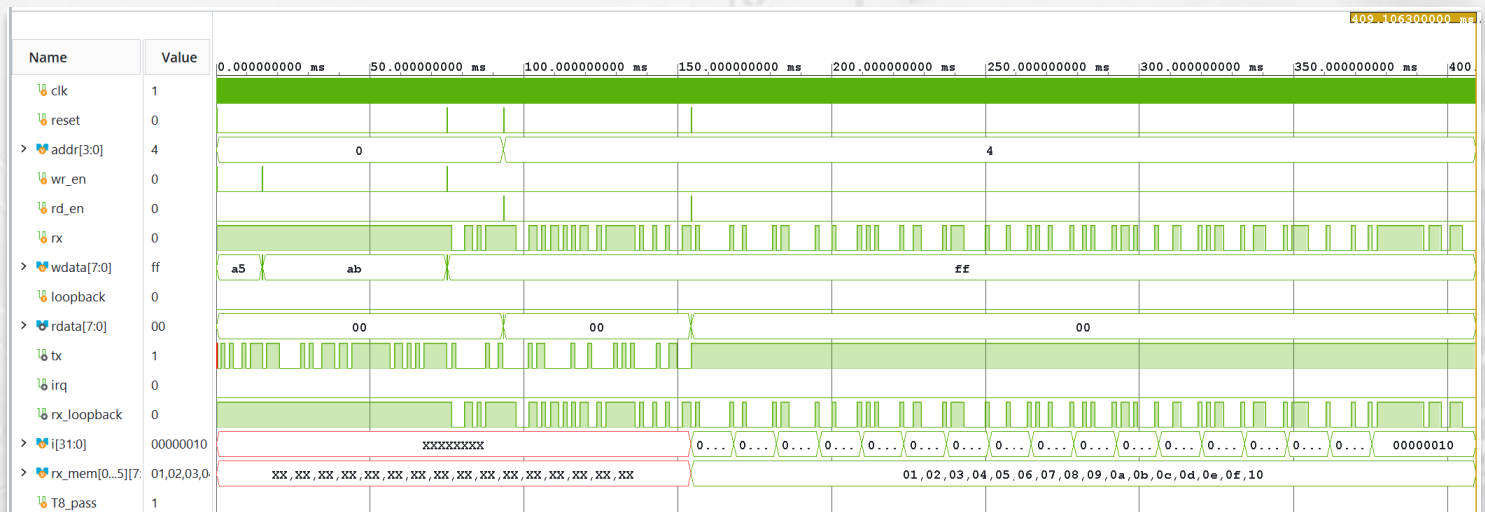


Figure 6.8: T8 Bad Stop Bit — Vivado Simulation Waveform

6.9 T9 — Register Access

Objective

Verify read and write operations for the UART registers.

Procedure

The CONTROL, BAUD_DIV, and STATUS registers are accessed through the host register interface and their returned values are checked.

Expected Result

All register operations return the expected values according to the defined register map.

Result: PASS 

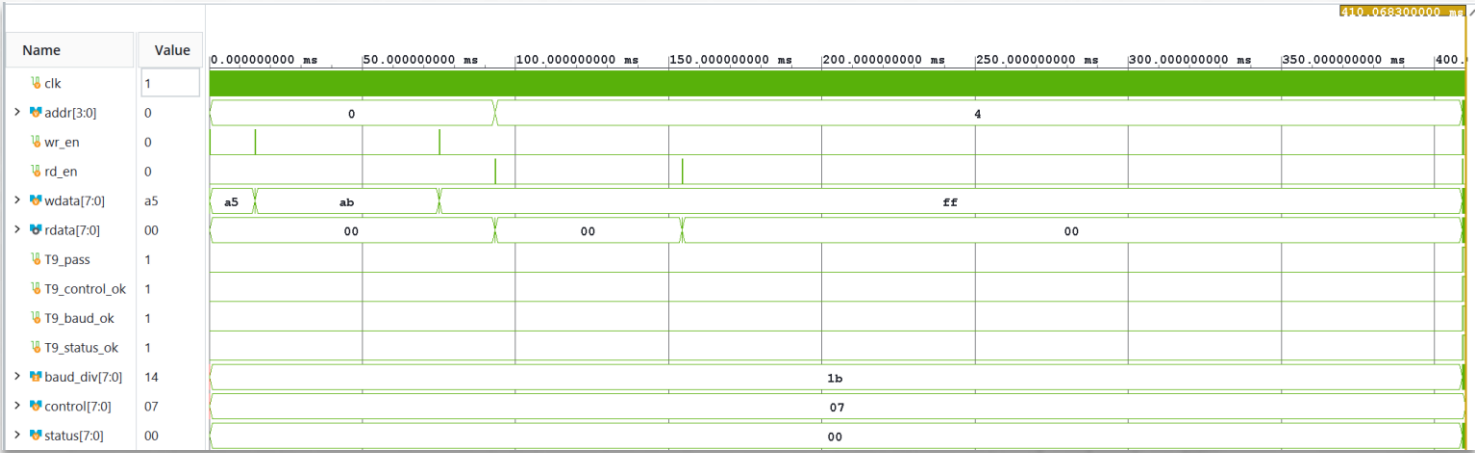


Figure 6.9: T9 Register Access — Vivado Simulation Waveform

6.10 T10 — Baud Scaling

Objective

Verify that changing BAUD_DIV changes the UART transmission timing proportionally.

Procedure

The same byte is transmitted using two different divisor values, BAUD_DIV = 10 and BAUD_DIV = 20, and the resulting transmission periods are compared.

Expected Result

Increasing BAUD_DIV increases the UART bit period proportionally.

Result: PASS

✓

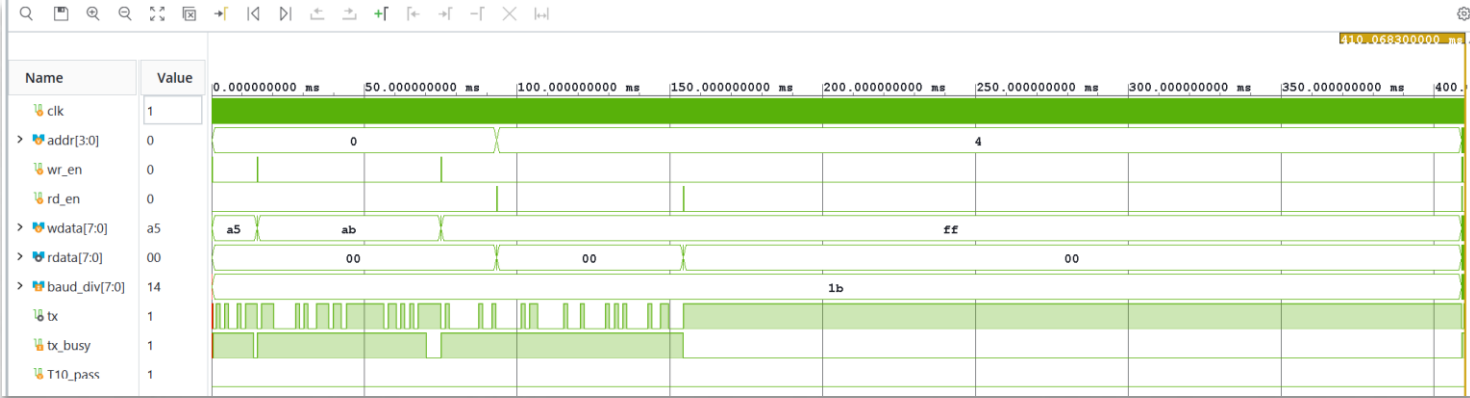


Figure 6.10: T10 Baud Scaling — Vivado Simulation Waveform

6.11 T11 — Full-Duplex Loopback

Objective

Verify simultaneous UART transmission and reception through a loopback connection.

Procedure

The TX output is connected internally to the RX input using the loopback path, and 8'hA5 is transmitted.

Expected Result

The transmitted byte is received correctly and becomes available through the RX FIFO and RX_DATA register.

Result: PASS

✓

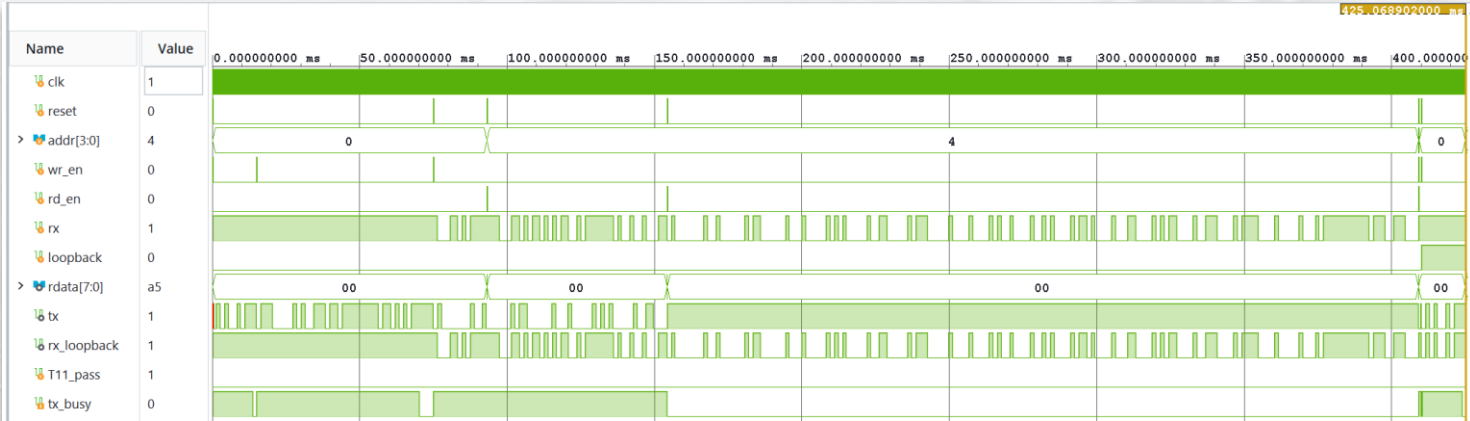


Figure 6.11: T11 Full-Duplex Loopback — Vivado Simulation Waveform

6.13 Verification Summary

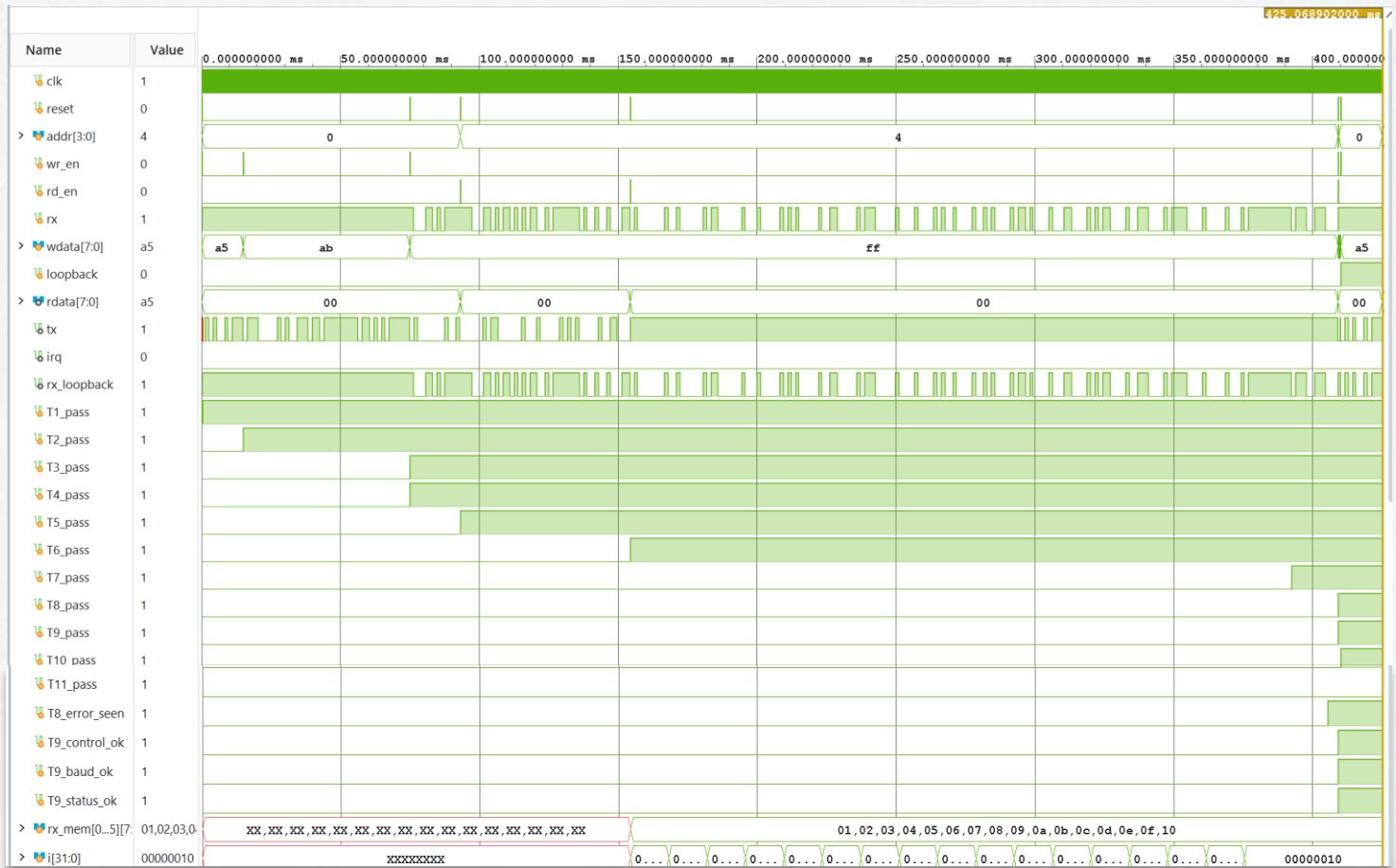


Figure 6.13: Full Waveform — Vivado Simulation Waveform

With a TCL Console Message:

```
=====
                        UART TESTBENCH RESULTS
=====

T1  RESET                : PASS
T2  SINGLE-BYTE TX       : PASS
T3  MULTIPLE-BYTE TX     : PASS
T4  TX FIFO FULL         : PASS
T5  SINGLE-BYTE RX       : PASS
T6  MULTIPLE-BYTE RX     : PASS
T7  RX FIFO FULL         : PASS
T8  BAD STOP BIT         : PASS
T9  REGISTER ACCESS      : PASS
T10 BAUD SCALING         : PASS
T11 FULL-DUPLEX          : PASS
=====
```

Figure 6.14: UART Testbench Results — Vivado Simulation Console

The simulation results confirm that all eleven defined verification tests were successfully completed. The UART Controller demonstrated correct operation of the transmission and reception paths, FIFO buffering, register interface, frame-error detection, programmable baud-rate control, and full-duplex loopback communication.

All test cases returned a **PASS** result, confirming the functional behavior of the integrated RTL design under the defined verification scenarios.



RTL Schematic

The RTL schematic illustrates the hardware structure and interconnection of the implemented UART Controller. It shows the main functional modules and the signal paths between the host register interface, TX and RX FIFOs, UART transmitter, UART receiver, and baud-rate generator.

The schematic provides a structural view of the complete design and complements the module-level descriptions presented in the previous sections. It also shows how the transmit and receive data paths are integrated within the top-level `uart_top` module.

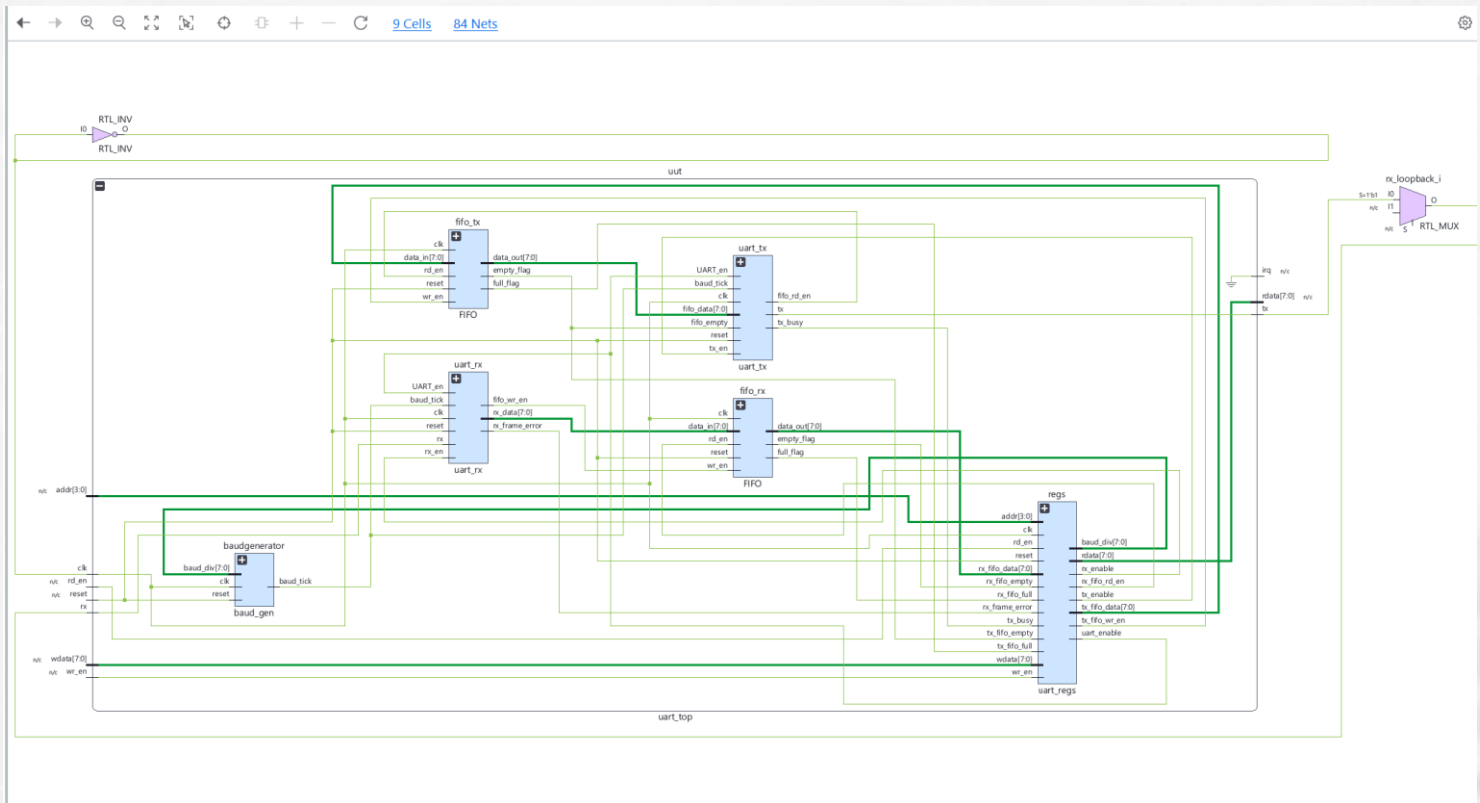


Figure 7.1: UART Controller RTL Schematic — Vivado Elaborated Design

Conclusion and Future Improvements

The UART Controller was successfully designed and implemented using synthesizable Verilog HDL. The design integrates UART transmission and reception, independent 16×8 TX and RX FIFOs, programmable baud-rate generation, and a memory-mapped register interface.

The complete RTL design was integrated within the `uart_top` module and verified through system-level simulation. The verification process covered reset operation, UART transmission and reception, FIFO boundary conditions, frame-error detection, register access, baud-rate scaling, and full-duplex loopback communication. **All eleven defined verification tests passed successfully**, confirming the functional behavior of the implemented design.

The current implementation provides the required Phase 1 UART functionality. Future improvements can extend the controller with additional features such as **parity support, configurable data and stop-bit formats, interrupt generation, 16550-compatible functionality, configurable FIFO trigger levels, and enhanced oversampling techniques.**

Overall, the project demonstrates the complete RTL design flow of a modular UART peripheral, from individual module implementation and integration to functional verification.